ABAP CODING CONVENTION

VERSION 1.5

RESTRICTED
Copyright © 2023 FPT All Rights Reserved.

VERSION CONTROL

Prepared by:	Technical Team
Date:	01/12/2022
Reviewed by:	Line Manager
Date:	28/12/2022
Approved by:	Line Manager
Date:	28/12/2022

VERSION HISTORY

Date	Version	Author	Description
13/01/2010	0.01	ERP.SAP	First creation
07/06/2016	1.0	ERP.SAP	1st release
01/09/2020	1.1	ERP.SAP	Minor updates
11/12/2021	1.2	ERP.SAP	Update conventions relating to Classes development
28/12/2022	1.3	ERP.SAP	Minor updates
09/01/2023	1.4	ERP.SAP	Add new guidelines
30/01/2023	1.5	ERP.SAP	Add new guidelines for RAP

Copyright © 2023 FPT All Rights Reserved.

TABLE OF CONTENT

ABAP CODING CONVENTION **1**

VERSION CONTROL **2**

VERSION HISTORY **2**

TABLE OF CONTENT **3**

1. OVERVIEW **6**

1.1 Description 6
1.2 Target Audience 6

2. GLOSSARY **7**

3. BASIC CONCEPT / ASSUMPTION **8**

3.1.1 Master Language 8
3.1.2 Program Layout 8
3.1.3 Using character 8
3.1.4 Pretty Printer 8
3.1.5 ABAP Editor Settings 9

4. NAMING CONVENTION **10**

4.1 ABAP Object naming rule 10
4.1.1 Naming for development objects 10
4.1.2 Name Space 10
4.1.3 Modules Identifiers 10
4.2 Other naming rules 12
4.3 Variables naming rules 12
4.3.1 Selection Screen 12
4.3.2 GUI 13
4.3.3 Program 13
4.3.4 Classes, Interfaces and Methods 14
4.4 Coding convention 17
4.4.1 Access to a file 17
4.4.2 Text element 17
4.4.3 Program annotation 18
4.4.4 Program header 19
4.4.5 Source code tracking 21

5. ABAP SYNTAX **23**

5.1 Basic Syntaxes 23
5.1.1 ABAP Statement 23
5.1.2 Logical Conditions 23
5.1.3 Assignment Command 23
5.1.4 Operation Statement 23
5.2 Shared function 24
5.3 Declare variable 24
5.4 Declare constant 24
5.5 Check source code 24
5.6 Table Maintenance 26
5.6.1 Nested Selects 26
5.6.2 SELECT... ENDSELECT 26

Copyright © 2023 FPT All Rights Reserved.

5.6.3 SELECT SINGLE and SELECT UP TO 1 ROWS 26
5.6.4 SELECT FOR ALL ENTRIES 26
5.6.5 SELECT and SELECT * 26
5.6.6 SELECT ... ORDER BY 26
5.6.7 INSERT, UPDATE, DELETE 26
5.6.8 Lock data 26
5.6.9 SY-SUBRC checks 26
5.7 Subroutines 27
5.7.1 Creating Subroutines 27
5.8 Methods 27
5.8.1 Methods Call 27
5.9 Internal table 27
5.9.1 Work area of internal table 27
5.9.2 Avoiding LOOP IN LOOP (Nested Loop) 28
5.9.3 Improving LOOP IN LOOP (Nested Loop) with Parallel Cursor 28
5.9.4 Reading a record in an internal table 29
5.9.5 Release memory 29
5.10 Report Programs 29
5.11 Background program 30
5.12 Error detected 30
5.13 Exception (Error Handling) 30
5.14 ABAP Memory 31
5.15 Obsolete 31

6. CORE DATA SERVICES (CDS) 32

6.1 CDS Naming Convention 32
6.1.1 General Rule 32
6.1.2 Prefixes and Suffixes 32
6.1.3 Field Names 34
6.1.4 DCL Source (Access Control) 35
6.1.5 Metadata Extension 35
6.1.6 BOPF Naming Convention 35
6.2 CDS General Guideline 35
6.2.1 Association for hierarchies, texts and values helps 35
6.2.2 Formatting 36
6.2.3 View Extension 36
6.2.4 UNION 36

7. ABAP RESTFUL APPLICATION PROGRAMMING MODEL (RAP) 37

7.1 RAP Naming Convention 37
7.1.1 ABAP Dictionary Objects 37
7.1.2 CDS Entity 37
7.1.3 Behavior Definition 37
7.1.4 Metadata Extension 37
7.1.5 Service Definition 38
7.1.6 Service Binding 38
7.1.7 Behaviour Pool 38
7.1.8 Handler and Saver Classes 38
7.2 RAP General Guideline 39
7.2.1 Required CDS Annotations 39
7.2.2 Use STRICT 39
7.2.3 Handling of Messages. 39
7.2.4 Passing Parameter to RAP Actions. 41
7.2.5 Using Virtual Elements in CDS Projection Views 42
7.2.6 RAP Custom Entity and Queries 43

Copyright © 2023 FPT All Rights Reserved.

7.2.7 Other RAP Guidelines form Standard Document of SAP 46

Copyright © 2023 FPT All Rights Reserved.

1. OVERVIEW

1.1 Description

This document aims to define the coding convention for ABAP, including the naming rules for variables, annotation, program convention, etc.

1.2 Target Audience

ABAP development team, Code Reviewers, Quality Control.

Copyright © 2023 FPT All Rights Reserved.

2. GLOSSARY

Term	Description
ABAP	Advanced Business Applications Programming
IDOC	Intermediate Document
SQL	Structured Query Language
BDC	Batch Data Communications
BAPI	Business Applications Programming Interface
RICEFW	Reports, Interfaces, Conversions, Extensions, Workflow

Copyright © 2023 FPT All Rights Reserved.

3. BASIC CONCEPT / ASSUMPTION

In principle, SAP ABAP Help has been described in the company to follow the defined constraints. Instructions have been deprecated or prohibited by SAP, also holds in this project.

SAP ABAP Help: https://help.sap.com/doc/abapdocu_latest_index_htm/latest/en-US/index.htm

3.1.1 Master Language

Set the Master language to English (EN). (absolutely adhere rigidly)

(When you develop, the logon language must be English.)

3.1.2 Program Layout

All ABAP programs should have an appropriate template for code readability and order. See section 3.4.4 for further details.

As a general rule, the programs should be divided into three major sections namely:

- Data Collection – This is the section that retrieves the data from relevant sources such as Database tables, files from Application and Presentation servers, IDOCs, etc.
- Data Processing – This is the section where you process the raw data from the previous section (Data Collection).
- Data Output – This is the section where you return the output or information to the user such as displaying a list, message, form, etc.

It is preferred that these sections be separated through Subroutines or Methods.

3.1.3 Using character

3.1.3.1 Source code

- Use only the alphanumeric character capital letter
- Use only the following characters for the item name

0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ_

3.1.3.2 Comments

- Write the comment in English
- Put the comments on Variables, PARAMETERS, SELECT-OPTIONS, Events and Subroutines

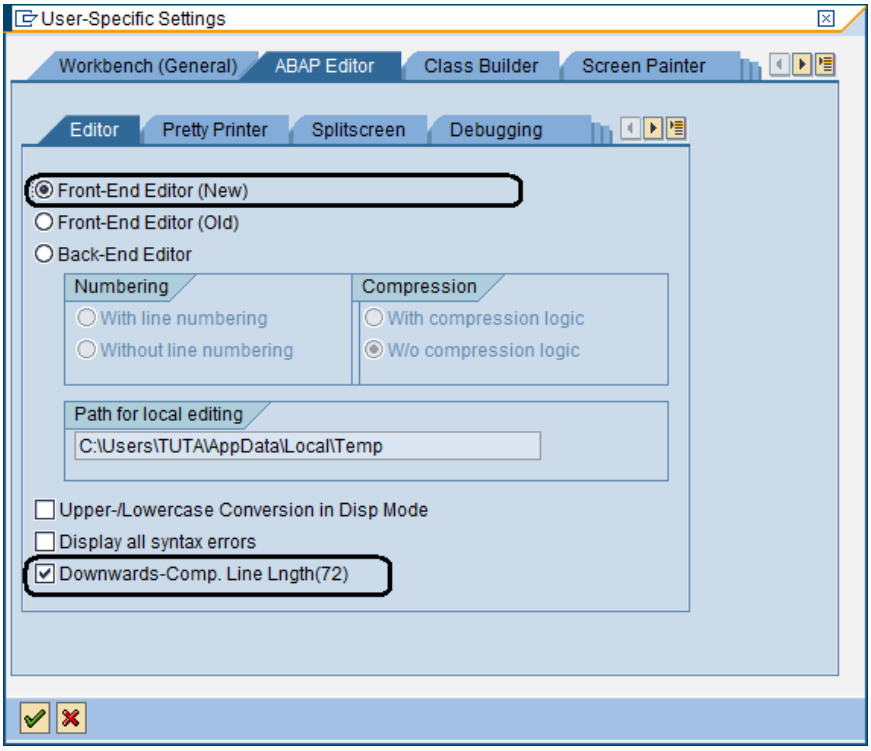
3.1.4 Pretty Printer

Pretty printer should be used to indent the code properly. See the next section [ABAP Editor Settings] for the Pretty Printer settings.

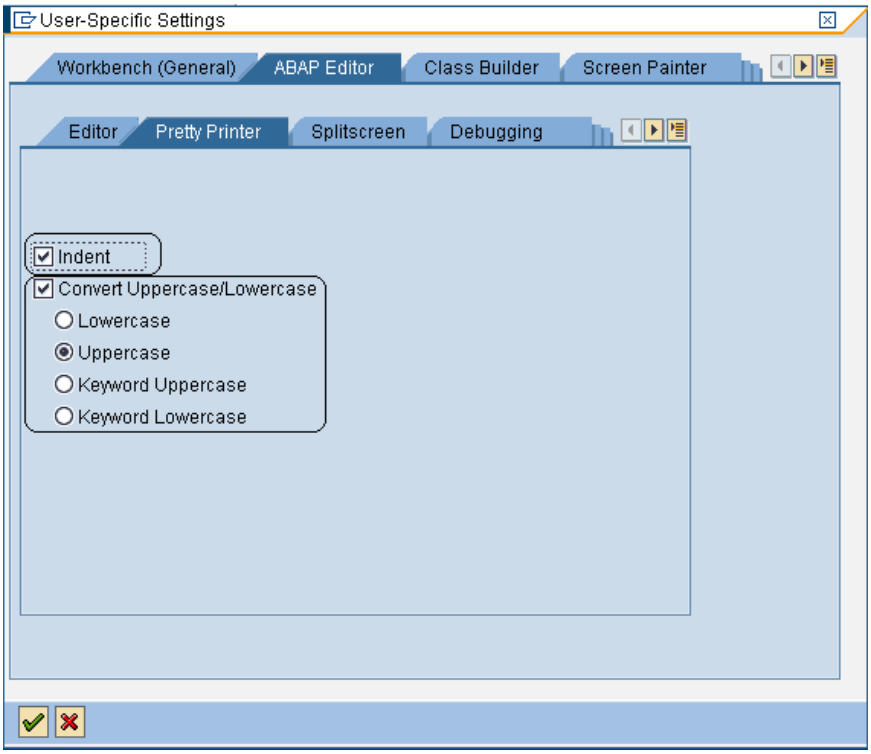
Copyright © 2023 FPT All Rights Reserved.

3.1.5 ABAP Editor Settings

SE38 / SE80 → Utilities → Settings...



Select Tab Pretty Printer



Copyright © 2023 FPT All Rights Reserved.

4. NAMING CONVENTION

4.1 ABAP Object naming rule

4.1.1 Naming for development objects

When you choose names for development objects, you should:

- Use **English** names
- Use **glossary terms** when possible

  For example, ZCL_COMPANY_CODE instead of BUKRS.
- In compound names, use the **underscore character (_) as a separator**. Since names are not case-sensitive, this is the only character you can use to separate names.

  For example, ZCL_COMPANY_CODE, ZCL_GENERAL_LEDGER_ACCOUNT.
- Names **should describe the action**, not the implementation of the action.

  For example, PRINT_RECTANGLE, not RECTANGLE_TO_SPOOL.

4.1.2 Name Space

No.	ID	Text	Description
1	/FPT/	FPT Name Space	For FPT Template Objects and FPT Solutions
2	Z*	Common Name Space	For Developing in common name space
3	Y*	Testing Common Name Space	For Test Programs or Training Programs

4.1.3 Modules Identifiers

Module / Submodule		Module Name	Module Identifier
Global (Cross Application)		CA	CA
Financial Accounting	**Global**	FI	FI
	General Ledger Accounting	FI-GL	GL
	Account Payable	FI-AP	AP
	Account Receivable	FI-AR	AR
	Asset Accounting	FI-AA	AA

Copyright © 2023 FPT All Rights Reserved.

	Treasury and Risk Management	FI-TRM	TM
	Profit Center Accounting	EC-PCA	PC
Controlling	**Global**	CO	CO
	Cost Center Accounting	CO-CCA	CC
Financial Supply Chain Management	**Global**	FSCM	FS
	Cash Management	FSCM-CM	CM
	Receivable Accounting	FSCM-COL	CL
	Liquidity Planner	FSCM-LP	LP
	Cash Flow	CF	CF
Local Consolidation		EC-CS	CS
Logistics	**Global**	LO	LO
	Sales Management	SD	SD
	Material Management	MM	MM
HR-TV		HR-TV	TV
BW		BW	BW
Business Planning and Consolidation		BPC	BP
ALE (Application Link Enabling)		ALE	AL
AIS (Audit Information System)		AIS	AI
Shared Service Framework		SSF	SS
Basis		BC	BC

Copyright © 2023 FPT All Rights Reserved.

4.2 Other naming rules

Other specific naming rules will be set by discussion with customer. Project should defined a separate Naming Rule File after the disscussion.

For example, intead of using English names, customers may prefer to use Module Identifier and Numbering like the example below.

Digit	**1**	**2**	**3-4**	**5-11**
Content Set	Fixing	Program Type	Module Component	Name
	"Z"	"R" : Report "B" : Batch	<Identifier defined by project rules> e.g., SD, MM, FI, AC, AS, QM, PP, etc.	###### 6 numbers that is registered in Project Program List as Program ID

Result: ZRSD000001, ZBMM000012

* This is example.

4.3 Variables naming rules

When you name variables, you should

- Use the underscore character (_) as a separator. Do not use "-" to avoid conflict.
- Subroutine, function module are considered as Local Parameters.

4.3.1 Selection Screen

Category	**Naming**		**Description**
	Global	**Local**	
PARAMETERS (Field)	N/A	P_*	Parameters are only up to 8 characters.
PARAMETERS (Checkbox)	N/A	CB_*	Parameters are only up to 8 characters.
PARAMETERS (Radio Button)	N/A	RB_*	Parameters are only up to 8 characters.
Radio button group	N/A	RG*	Radio button group is up to 4 characters only. Example: RG01, RG99
SELECT-OPTIONS	N/A	S_*	Select Options can only be up to 8 characters.
Block	N/A	BL_*	

Variable name should be the same as table field or data element.

Example: P_BUKRS TYPE BUKRS to define company code.

Copyright © 2023 FPT All Rights Reserved.

(P_COMPANY_NAME is not possible because Selection Screen Parameters and Select-Options support only 8-characters names)

4.3.2 GUI

Category	Naming rule	Description
Box	GRP*	If the group heading is not fixed, you should follow the program naming rule.
Push Button	CMD*	If the textual label is not fixed, you should follow the program naming rule.
Function Codes	ZXXX	Where XXXX is a four letter description
Table Control	TCL*	
Tab Control	TAB*	
Tab command	TB_*	If the textual label is not fixed, you should follow the program naming rule.
Subscreen Area	SUB*	
Custom Control	CUS*	
Screen field	*	The screen field should refer to structure, table or view to ABAP dictionary.
Combo box	*	Use Listbox with key.
Group box	G*	* is the sequential number to define the customizing group of the field. Example: group box of General Address in Customer Master Data

4.3.3 Program

Category	Global	Local
Constant (CONSTANTS)	GC_*	LC_*
General variable	GV_*	LV_*
Statics	N/A	ST_*
Flag variable	GV_FLG_*	LV_FLG_*
Counter variable	GV_CNT_*	LV_CNT_*

Copyright © 2023 FPT All Rights Reserved.

Category	Global	Local
Ranges	GR_*	LR_*
Macros	GM_*	LM_*
Internal table	GT_*	LT_*
Work area / Structure	GS_*	LS_*
Structure type	GTY_*	LTY_*
Type (table type)	GTY_T_*	LTY_T_*
Field symbol	<GFS_*>	<LFS_*>

Variable name should be the same as table field or data element.

Example: GV_BUKRS TYPE BUKRS to define company code.

4.3.4 Classes, Interfaces and Methods

Classes, Interfaces

Global Class in the class library	ZCL_<class name> The class name should consist of singular nouns. ZCL_COMPANY_CODE, ZCL_GENERAL_LEDGER_ACCOUNT
Global Interfaces in the class library	ZIF_<interface name> The same naming convention as for classes. ZIF_STATUS_MANAGEMANT, ZIF_CHECKER
Local classes in programs (recommendation)	LCL_<class name> The class name should consist of singular nouns. LCL_TREE_MANAGEMENT
Local interfaces in programs (recommendation)	LIF_<interface name> The same naming convention as for classes. LIF_PRINTER

Copyright © 2023 FPT All Rights Reserved.

Methods

All Methods should be named in 61 characters. The Method name should be a verb phrase. You can split between words by an underscore _. The only exception to this would be the CONSTRUCTOR method.

Normal Methods	<any name must begin with verb>
Attribute access	SET_<attribute name>, GET_<attribute name> Methods that access attributes of any kind should have the prefix GET_ or SET_. GET_STATUS, SET_USE_COUNT
Events Handling Methos	ON_<event name> Methods that handle events should begin with ON, followed by the name of the event that they handle. ON_BUTTON_PUSHED, ON_BUSINESS_PARTNER_PRINTED
Methods that perform type conversions	AS_<new type> AS_STRING, AS_ISOCODE
Methods that return a Boolean value	IS_<adjective> IS_OPEN, IS_EMPTY, IS_ACTIVE These methods may not return any exceptions.
Check methods	CHECK_<objective> CHECK_AUTHORIZATION, CHECK_PROCESS_DATE

Example: GET_STATUS, CREATE_ORDER, DETERMINE_PRICE

```
METHOD GET_DATA.
    ...
ENDMETHOD.
```

Method Parameters

The parameters are regarded from the point of view of the method that implements them:

Copyright © 2023 FPT All Rights Reserved.

Category	Naming rule
IMPORT	IM_*
EXPORT	EX_*
CHANGING	CH_*
RETURNING	RE_*

Events

All Events should be named in 61 characters. The **Event names** should have the form <noun>_<participle>.

Events	<noun>_<participle>.

Example: BUTTON_PUSHED, COMPANY_CODE_CHANGED, BUSINESS_PARTNER_PRINTED

4.3.5 Subroutine

Category	Local
General Variable	*
Flag Variable	*_FLG_*
Counter Variable	*_CNT_*
Internal table	*_T_*
Structure (work area)	*_S_*
Undefined type	*_P_*

Variable name should be the same as table field or data element.

Example: I_BUKRS TYPE BUKRS to define company code.

Category	Naming rule
USING	I_*
CHANGING	C_*

Copyright © 2023 FPT All Rights Reserved.

4.3.6 Function Module

Category	Naming rule
IMPORT	I_*
EXPORT	E_*
CHANGING	C_*
TABLES	T_*

Variable name should be the same as table field or data element.

Example: I_BUKRS TYPE BUKRS to define company code.

4.4 Coding convention

4.4.1 Access to a file

When you want to access a file located in the server, you should state the logical path to avoid the dependency of operating system.

Create the Logical Path in TCODE FILE.

4.4.2 Text element

This section describes how to design texts. You should use the text element instead of fixed text.

Example: Instead of fixing a message text like “Data cannot be saved”, create a text element TEXT-001 and enter the description “Data cannot be saved”. By this way, you have the option to translate the text in different languages if required.

4.4.2.1 Text element category

Category	Description	Usage
List header/row header	To design the report title and column heading.	Do not use this function. Use the key word WRITE and event TOP-OF-PAGE to design the header.
Selection text	To display the documentation help of the field in the selection screen	To display the documentation help of the field in the selection screen
Text symbol	Do not hard-code texts in your program. Text symbols should be used in the program and should be numbered from 001.	To display the message, report title, replace hardcoded literals...

Copyright © 2023 FPT All Rights Reserved.

4.4.3 Program annotation

Annotation aims to explain the functionality, usage or modification.

4.4.3.1 Program annotation rule

- Annotation should be English.
- Use * to note at the beginning of the line.
- Use “ to note in the middle of line.
- Use the below template for Annotations per section.

Template.docx

4.4.3.2 Annotation for variables declaration

Annotation should be in right side of the variables. In the case of the annotation is too long, it can be placed in the previous line.

Example:

```
DATA:

  LV_GPART_CNT TYPE I.   “ Number of Business Partner

DATA:

* Account Balance Of Business Partner

  LT_BUT000 TYPE STANDARD TABLE OF BUT000.
```

4.4.3.3 INCLUDE

Annotation should be in right side of the INCLUDE. In the case of the annotation is too long, it can be placed in the previous line.

Example:

```
INCLUDE  ZINRM01_TOP.      " Global data
INCLUDE  ZINRM01_F01.      " Subroutine
INCLUDE  ZINRM01_O01.      " Process before output
INCLUDE  ZINRM01_I01.      " Process After Input
```


Copyright © 2023 FPT All Rights Reserved.

4.4.3.4 SELECT function

To describe the functionality of SELECT function.

```
**-------------------------Select function-----------------------------*
CASE OPT;
  WHEN ‘DISP'.
  WHEN ‘EDIT'.
END CASE.
**-------------------------Select function-----------------------------*
```

4.4.3.5 Call Subroutine or Function Module

Annotation should be placed in the previous line.

Example: Function module

```
* Open BDC session
CALL FUNCTION 'BDC_OPEN_GROUP'
EXPORTING....
```

Subroutine

```
* Open batch input session
PERFORM BDC_OPEN.
```

4.4.3.6 Other annotations

Annotation is mandatory for IF command or complex ABAP command.

Example:

```
DATA      GC_DISP(4) TYPE C VALUE ‘DISP’.           “ Display mode
IF OPT = GC_DISP.               " when the option is a display mode
   LV_CNT = LV_CNT + 1.    " increase the counter
```

4.4.4 Program header

4.4.4.1 Header of the main program

Annotation required at the header of the main program.

```
**-------------------------------------------------------*
** Program ID:   XXXXXX
** Program name: XXXXXXXXXXXXXXX
** RICEFW ID:    RM1.2_001
** Created by:
```


Copyright © 2023 FPT All Rights Reserved.

```
** Created date:
** Content explanation: Content line 1
**                      content line
**-------------------------------------------------------*
** <Modification tracking>
** Modification number: XXXXXXXXX
** Modification day:
** Modification reason: (after release in testing environment)
**
**-----------------------------------------------------------*
```

4.4.4.2 Header of program INCLUDE

Header of program INCLUDE should be noted as below.

```
*&---------------------------------------------------------------------*
* INCLUDE <Include Name>
*&---------------------------------------------------------------------*
* <Content Explanation>
*&---------------------------------------------------------------------*
```

4.4.4.3 Header of Subroutine

Header of subroutine should be noted as below.

```
*&---------------------------------------------------------------------*
*& Form <SUBROUTINE NAME> (Uppercase)
*&---------------------------------------------------------------------*
*     <Subroutine processing detail>
*----------------------------------------------------------------------*
*     --> I_IN      :     <Input meaning>
*     <-> CT_TEXT   :     <In/Output meaning>
*     <-- C_OUT     :     <Output meaning>
*----------------------------------------------------------------------*
FORM <SUBROUTINE NAME>.
```

4.4.4.4 Header of Function module

Header of function module should be noted as below.

Copyright © 2023 FPT All Rights Reserved.

```
**-------------------------------------------------------*
** Function ID:   XXXXXX
** Function name: XXXXXXXXXXXXXXX
** RICEFW ID:     RM1.2_001 (RICEFW item of function module)
** Created by:
** Created date:
** Content explanation: Content line 1
**                      content line 2
**-------------------------------------------------------*
** <Modification tracking>
** Modification number: XXXXXXXXX
** Modification day:
** Modification reason:
**
**-----------------------------------------------------------*
```

4.4.5 Source code tracking

After being released to test/production environment, the program or the Function Module may have bugs or change request. You should track these modifications.

4.4.5.1 Add note to main program's header or function module

```
**-------------------------------------------------------*
** <Modification tracking>
** Modification number: A00000001 (BUG ID for Change Request ID)
** Modification date:   12/12/2009
** Modification reason: Modification content line 1
**                      Modification content line 2
**-----------------------------------------------------------*
** <Modification tracking>
** Modification number: A00000002 (BUG ID for Change Request ID)
** Modification day:    15/12/2009
** Modification reason: Modification content 2 line 1
**                      modification content 2 line 2
**-----------------------------------------------------------*
```


Copyright © 2023 FPT All Rights Reserved.

4.4.5.2 Add or change a line

To add or change a line, you should note:

Example:

Before modification:

```
WRITE :/01 TEXT-001.
```

After modification:

```
* <Modification Number> <Modification Date> <Modification By> - START
* WRITE :/01 TEXT-001.    “ DEL XXXXXXXX
  WRITE :/01 TEXT-002.    “ ADD XXXXXXXX
* <Modification Number> <Modification Date> <Modification By> - END
```

Do not delete any line. If a significant modification is required, you should create a new version and delete the old one.

XXXXXXXX is the description of the annotation

Copyright © 2023 FPT All Rights Reserved.

5. ABAP SYNTAX

5.1 Basic Syntaxes

5.1.1 ABAP Statement

Every ABAP statement should be on a single line. In case of the SELECT statement, every clause like FROM, WHERE etc should be on a different line. A maximum of 3 Field Selection should be written on each line.

Example :

```
SELECT matnr mtart meins
brgew ntgew gewei
  FROM mara
    INTO TABLE gt_mara
WHERE    matnr =  p_matnr
    AND lvorm <> gc_X.
```

5.1.2 Logical Conditions

- Always use the symbolic operator (i.e. <>, = , <=, etc) instead of the letter operator (i.e. EQ, GE, GT, LE, etc) in ABAP and SQL conditions.
- Each AND or OR statement should be on separate lines.

  Examples :

  1)
  ```
  IF  GV_bukrs =  p_bukrs
  AND GV_werks <> p_werks.
  ```

  2)
  ```
  SELECT bukrs
    FROM T001
      INTO TABLE gt_bukrs
  WHERE bukrs = p_bukrs.
  ```

5.1.3 Assignment Command

Use "=" for value assignment

※ Do not use “MOVE ~ TO ~” instruction, except for MOVE-CORRESPONDING

5.1.4 Operation Statement

Do not use following statements: ADD，SUBTRACT，MULTIPLY，DIVIDE

Copyright © 2023 FPT All Rights Reserved.

Abbreviate the COMPUTE statement, and Unify operator

- Add : A = B + C
- Subtract : A = B – C
- Multiply : A = B * C
- Divide : A = B / C
- Quotient of Division(Integer) : A = B DIV C
- Remainder of Division : A = B MOD C
- Index Calculation : A = B ** C

5.2 Shared function

If there are universe of different functions which are shared in different programs, you should use INCLUDE or Function Module.

5.3 Declare variable

Always use TYPE statement to declare variable. In some special case, use LIKE statement.

E.g.,

```
DATA: GV_COMPCODE TYPE BSID-BELNR.
```

5.4 Declare constant

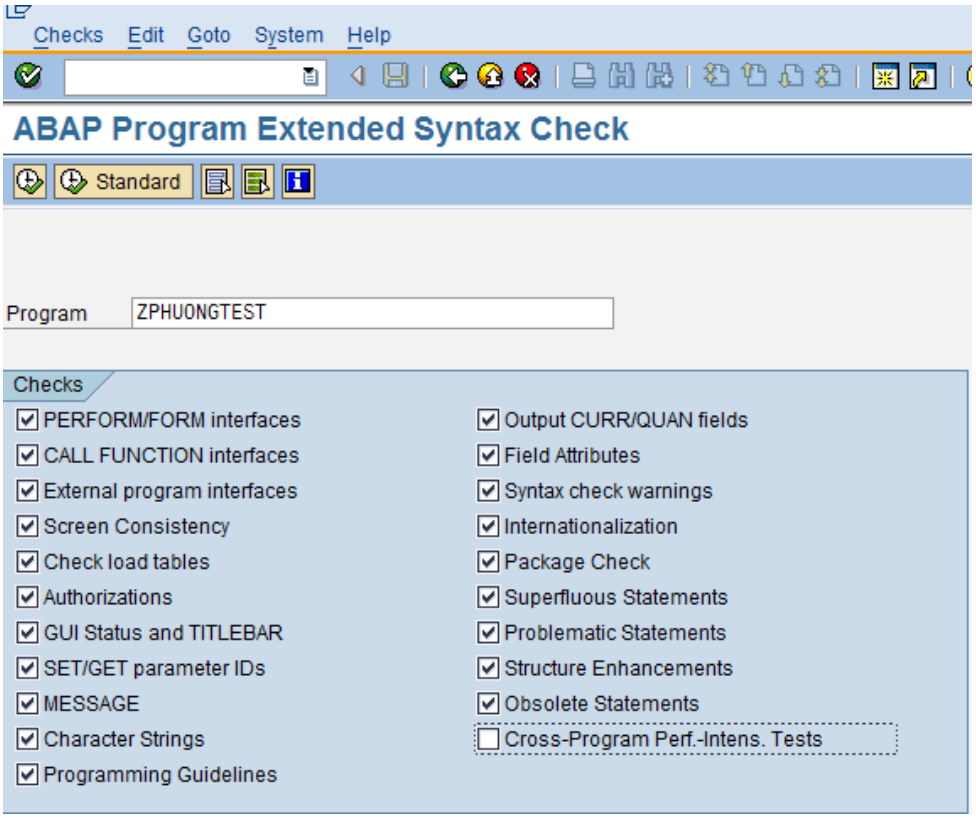
Use CONSTANTS statement to declare constant. Use this declaration in the program instead of the constant value.

5.5 Check source code

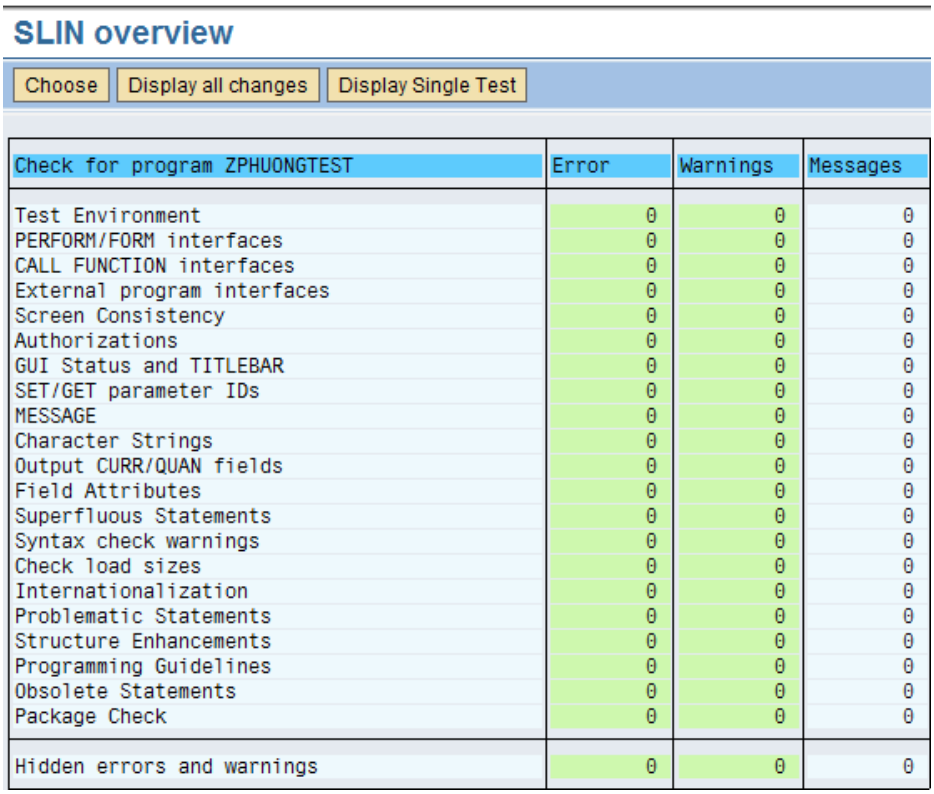
After finishing the source code, you should run the function of source code checking and correct any warning.

From ABAP Editor (SE37, SE38, SE80), run Program->Check->Extended Program Check.

Copyright © 2023 FPT All Rights Reserved.



Tick all checkboxes, except the last one.



Check for program ZPHUONGTEST	Error	Warnings	Messages
Test Environment	0	0	0
PERFORM/FORM interfaces	0	0	0
CALL FUNCTION interfaces	0	0	0
External program interfaces	0	0	0
Screen Consistency	0	0	0
Authorizations	0	0	0
GUI Status and TITLEBAR	0	0	0
SET/GET parameter IDs	0	0	0
MESSAGE	0	0	0
Character Strings	0	0	0
Output CURR/QUAN fields	0	0	0
Field Attributes	0	0	0
Superfluous Statements	0	0	0
Syntax check warnings	0	0	0
Check load sizes	0	0	0
Internationalization	0	0	0
Problematic Statements	0	0	0
Structure Enhancements	0	0	0
Programming Guidelines	0	0	0
Obsolete Statements	0	0	0
Package Check	0	0	0
Hidden errors and warnings	0	0	0

Correct any warning.

Copyright © 2023 FPT All Rights Reserved.

5.6 Table Maintenance

Do not use INSERT, UPDATE, DELETE statement for standard table, use BAPI or Function module instead.

5.6.1 Nested Selects

The performance of nested SELECT loops is very poor. Hence, do not use nested SELECT, except the case of large data.

5.6.2 SELECT... ENDSELECT

ENDSELECT should only be used if the UP TO 1 ROWS condition is added.

5.6.3 SELECT SINGLE and SELECT UP TO 1 ROWS

Use SELECT SINGLE to read database records with primary key.

In the absence of the primary key, use UP TO 1 ROWS.

5.6.4 SELECT FOR ALL ENTRIES

You should check the following matters:

- Internal table should not be empty. If it is empty, then all rows will be retrieved.
- Ensure the retrieved records will contain no duplicates because SAP automatically removes any duplicates from the rest of the retrieved records.

5.6.5 SELECT and SELECT *

In general, use a SELECT statement specifying a list of fields instead of a SELECT * to reduce network traffic and improve performance

5.6.6 SELECT ... ORDER BY

Do not use ORDER BY to sort data. Data will be sorted in the internal table.

5.6.7 INSERT, UPDATE, DELETE

Use Open SQL to insert/update/delete, do not use Native SQL. Make sure that the program issues COMMIT WORK or ROLLBACK WORK statements after inserting, updating or deleting. Use COMMIT WORK if sy-subrc equals to zero, else, write ROLLBACK WORK.

5.6.8 Lock data

Set database lock before it receives change statements (**INSERTS, UPDATE, MODIFY, DELETE**) if it is required and evaluation is done before using the ENQUEUE and DEQUEUE functionality.

5.6.9 SY-SUBRC checks

SY-SUBRC should be checked after every SELECT Statement that requires processing based on success or unsuccessful data retrieval.

Copyright © 2023 FPT All Rights Reserved.

5.7 Subroutines

5.7.1 Creating Subroutines

Subroutines should be used for modularization of the code and improving code readability. When creating subroutines, add the prefix f_ to your form.

Example:

```
PERFORM f_add_counter.
```

5.8 Methods

5.8.1 Methods Call

Formulate static method calls without CALL METHOD.

Use the long form of the method call using CALL METHOD only for dynamic method calls.

Bad Example:

The following source code shows the long form of a static method call using CALL METHOD, which is no longer recommended.

```
...
CALL METHOD cl_class=>do_something
  EXPORTING
    some_input = value1.
...
```

Good Example:

The following source code shows the same method call as above, but as recommended, without CALL METHOD. If a method has only importing parameters, IMPORTING and CHANGING can be omitted, and also the EXPORTING addition. If it is a single importing parameter, its name can also be omitted.

```
...
cl_class=>do_something( EXPORTING some_input = value1 ).
```

5.9 Internal table

5.9.1 Work area of internal table

Do not use header line, use work areas or field-symbols. Do not use TABLE when specifying parameters of subroutine.

Example:

```
TYPES: BEGIN OF y_bseg,
```


Copyright © 2023 FPT All Rights Reserved.

```
                bukrs TYPE bseg-bukrs,
                belnr TYPE bseg-belnr,
                wrbtr TYPE bseg-wrbtr,
                END OF y_bseg.

DATA: gt_bseg TYPE STANDARD TABLE OF y_bseg,
            gs_bseg TYPE y_bseg.

PERFORM f_get_data USING gt_bseg.

FORM f_get_data USING t_bseg LIKE gt_bseg.
  SELECT bukrs
         belnr
         wrbtr
    FROM bseg
       INTO TABLE t_bseg
  WHERE bukrs = p_bukrs
       AND belnr  IN s_belnr.
ENDFORM.                    " F_GET_DATA
```

5.9.2 Avoiding LOOP IN LOOP (Nested Loop)

Avoid using nested LOOPs to improve performance.

Use SQL techniques like using FOR ALL ENTRIES or INNER JOIN.

5.9.3 Improving LOOP IN LOOP (Nested Loop) with Parallel Cursor

IF your business logic cannot void nested LOOP, try to improve it with Parallel Cursor technique.

For improved program performance, to determine which loop is better, and try to minimize memory load.

Specify the first index and break statement in inside Loop statement, try to minimum sequential search.

```
LOOP AT it_vbak ASSIGNING <lfs_vbak>.
  READ TABLE it_vbap TRANSPORTING NO FIELDS
       WITH KEY vbeln = <lfs_vbak>-vbeln
       BINARY SEARCH.
  IF sy-subrc EQ 0.
  LOOP AT it_vbap FROM sy-tabix  " << minimum sequential search
```


Copyright © 2023 FPT All Rights Reserved.

```
        ASSIGNING <lfs_vbap> .
    IF <lfs_vbap>-vbeln <> <lfs_vbak>-vbeln.
      EXIT.
    ENDIF.
*   Rest of the logic would go from here...
  ENDLOOP.
  IF <lfs_vbap>-kwmeng IS NOT INITIAL.
     " This would be next subsequent entry in the table IT_VBAP
  ENDIF.
  ENDIF.
ENDLOOP.
```

5.9.4 Reading a record in an internal table

Use BINARY SEARCH for better performance.

Explicit BINARY SEARCHES should only be used on sorted Standard Internal Tables.

SY-SUBRC should be checked after READ TABLE.

5.9.5 Release memory

All internal tables should be freed from memory if it is no longer used by succeeding processes of the program. To do this, use the FREE command.

Use Local Data instead of Global Data also help in memory management. Local Data will be deallocated automatically after going out of scope.

5.10 Report Programs

ALV is preferred than classical reports.

5.10.1 Output of report of amount of money item

Use CURRENCY option

5.10.2 Output of report of numeric item with unit

Use UNIT option

5.10.3 Output of report of amount of money & numeric item

Output right justified.

5.10.4 Output of common report header information

Use the common feature.

Copyright © 2023 FPT All Rights Reserved.

5.10.5 Error message in START-OF-SELECTION Event

Do not use message as TYPE: E.

Use DISPLAY LIKE 'E' option.

5.11 Background program

The message will be displayed on the Job Log Entries. The program will be terminated if the message type = E, A, X. Therefore, when the error type E, A, X happen, the program should log all other messages and terminate function have to be processed.

5.12 Error detected

5.12.1 Error detected in subroutine

Use SY-SUBRC command for error detected in subroutine.

5.12.2 Error detected in BAPI

Use RETURN command for error detected in BAPI and the appropriate action

(i.e. COMMIT WORK or ROLLBACK WORK) should be done where applicable.

5.12.3 Error detected in function module

Use EXCEPTION command for error detected in Function Module.

5.13 Exception (Error Handling)

As a rule, the judgment by the return code is indispensable.

However, describe the reason to the comment when error handling is unnecessary in the inquiry and the report function, etc.

Processing outside the typical example is described as follows.

5.13.1 Zero divide

When denominator is 0, do not execute the calculation processing and implement the exception handling matched to the requirement.

5.13.2 Numeric overflow

In all the total processing etc., implement the error correspondence by “TRY~CATCH” syntax when there is a possibility of the overflow.

Overflow : CX_SY_ARITHMETIC_OVERFLOW

Copyright © 2023 FPT All Rights Reserved.

5.13.3 Other

You must mount the error correspondence if necessary by using “TRY~CATCH” syntax when it can be handled the exception by exception class.

Overflow at SQL SUM : CX_SY_OPEN_SQL_DB

Overflow at transfer : CX_SY_CONVERSION_OVERFLOW

※Overflow at conversion of CHAR→PACK

5.14 ABAP Memory

Use the FREE command to clear Memory ID’s after use.

5.15 Obsolete

Do not use obsolete command in SAP ECC 6.0 and S/4HANA because they are noted as Error/Warning when running Program Extended Check.

Copyright © 2023 FPT All Rights Reserved.

6. CORE DATA SERVICES (CDS)

6.1 CDS Naming Convention

6.1.1 General Rule

CDS view names should consist of up to 4 elements, which are illustrated in the following example.

Example

ZC_ProfitAndLossQ_2

The naming elements in the example are broken up and explained in more detail in the following table.

Naming Elements	Example	Explanation
Prefix (obligatory)	ZC_	Indicates the view type (here: consumption view) and is separated from the core name with an underscore.
Semantic name (obligatory)	ProfitAndLoss	Based on business semantics and indicates what the CDS view is about (here: the movement of goods).
Suffix (optional)	Q	Defines the purpose of the view further. Here, we're dealing with an analytical query view.
Version number (optional)	_2	If there's more than one version of a view, subsequent versions receive a version number. The number is separated from the semantic name or suffix with an underscore.

6.1.2 Prefixes and Suffixes

The different VDM view types are easily recognized by their prefix.

SAP Standard Prefix	FPT Prefix	View Type

Copyright © 2023 FPT All Rights Reserved.

I_	ZI_	Basic interface view Composite interface view
C_	ZC_	Consumption view
R_	N/A	Basic restricted reuse views Composite restricted reuse views
P_	N/A	Private view
A_	ZA_	Remote API view
X_	N/A	View extends
E_	ZE_	Extension include view
F_	ZF_	Derivation function
D_	ZD_	Abstract entity
N/A	YI_, YC_, YE_, YA_	Test View or Training View

The following table summarizes the most common suffixes and the respective view types used in the VDM:

Suffix	View Type
Query, Qry, or Q	Analytical query view
Cube or C	Analytical cube view
Text, Txt, T	Provider for language-dependent text
TP	Transactional processing view
VH	Value help view

Copyright © 2023 FPT All Rights Reserved.

6.1.3 Field Names

Field names are based on business semantics to make their purpose more transparent. They typically fulfil the following criteria:

- Uniformity and avoidance of ambiguity:
  The same name for the same entity everywhere; different names for distinct entities.
- Legibility by using camel case: For example, **CostCenter**.
- Avoidance of abbreviations:
- Standardized abbreviations can be used if necessary to stay within the 30 characters limit.

The following table is a non-exhaustive list of different semantic field types with some examples.

Representation Term	Definition	Example Field Names
Identifier	A value identifying an instance of an object.	BankAccount (human-readable identifier) BankAccountUUID (technical identifier)
Code	A value from a range of possible values.	CorrespondenceLanguage TransactionCurrency
Indicator	A Boolean truth value (true or false).	OrderIsReleased
Amount	A monetary amount.	TaxAmount
Date	A calendar date.	GoodsArrivalDate
Time	A specific time of day.	ShiftDay1EndTime
Date and Time	A calendar date and a time on that date.	CreationDateTime
Quantity	A countable or measurable quantity.	InspectedProductQuantity
Text	Textual information.	GoodsLocationText
Duration	The duration between two points in time.	ServiceWorkDuration

Copyright © 2023 FPT All Rights Reserved.

6.1.4 DCL Source (Access Control)

DCL view names should be the same as the CDS View which it is granting access.

6.1.5 Metadata Extension

Metadata Extension names should be the same as the Consumption CDS View which it is annotating.

6.1.6 BOPF Naming Convention

6.1.6.1 Draft Table

We use suffix "_D" for Draft Table

Digit	1	2-14	15-16
Content Set	Prefix	Add-on table ID	Suffix
	"Z" or "Y"	ID of the add-on table as the source of the draft table.	"_D"

6.1.6.2 Interface ID

Adopted automatically suggested code when generating BOPF.

6.1.6.3 Determination, Validation and Action Class

Determination class	ZCL_D_<class name> The class name should consist of singular nouns.
Validation class	ZCL_V_<class name> The class name should consist of singular nouns.
Action class	ZCL_A_<class name> The class name should consist of singular nouns.

6.2 CDS General Guideline

6.2.1 Association for hierarchies, texts and values helps

- Associate dimension to dimension fields to take advantage of attributes, hierarchies, texts and values helps.

  For example:

  - Associate I_Customer to Customer field to enable texts and values helps.

Copyright © 2023 FPT All Rights Reserved.

- Associate I_GLAccount to GLAccount field to enable hierarchies, texts and values helps

6.2.2 Formatting

- If needed, cast custom fields to SAP data elements to get the right descriptions and formatting.
- Annotate Quantity and Amount fields with Unit of measures and Currency to ensure correct aggregation and presentation.

6.2.3 View Extension

- If you extend standard CDS view with custom fields then name custom fields starting with Z to avoid future naming collisions with SAP standard fields.

  For example: ZAdditionalField1

6.2.4 UNION

- Use UNION ALL instead of UNION where applicable to improve performance.

Copyright © 2023 FPT All Rights Reserved.

7. ABAP RESTFUL APPLICATION PROGRAMMING MODEL (RAP)

7.1 RAP Naming Convention

7.1.1 ABAP Dictionary Objects

Use a suffix for database tables in scenarios, in which multiple technical representations of the same semantic data is necessary, for example in draft scenarios.

Use the suffix.

- D_ for the draft database table.

Example: /FPT/MYTABLE_D or ZMYTABLE_D

7.1.2 CDS Entity

This naming convention for RAP bases mostly on CDS View Convention

SAP Standard Prefix	FPT Prefix	View Type
I_	ZIR_ /FPT/IR_	Root Entity view
I_	ZI_ /FPT/I_	Normal/Child Entity View
C_	ZCR_ /FPT/CR_	Root Projection View Or Root Custom Entity
C_	ZC_ /FPT/C_	Normal/Child Projection View

7.1.3 Behavior Definition

A behavior definition has always the same name as the root entity of the business object.

For example: /FPT/IR_SalesCredit, ZC_SalesCreditReport

7.1.4 Metadata Extension

A metadata extension has the same name as the CDS entity it relates to. If you use more than one metadata extension for one CDS entity, you can add a numbered suffix.

For example: /FPT/CR_SalesOrder, ZC_SalesCreditReport

Copyright © 2023 FPT All Rights Reserved.

7.1.5 Service Definition

Since a service definition - as a part of a business service - does not have different types or different specifications, there is (in general) no need for a prefix or suffix to differentiate meaning, but in FPT we add SD_ prefix to help documentation.

Example: /FPT/SD_SalesCredit, ZSD_SalesCreditReport

However, in use cases where no reuse of the same service definition is planned for UI and API services, the prefix may follow the rules of the service binding.

Example: /FPT/SD_SalesCreditAPI, ZSD_SalesCreditReportUI

7.1.6 Service Binding

Use the prefix.

- **UI_** if the service is exposed as a UI service.
- **API_** if the service is exposed as Web API.

Use the suffix.

- **_O2** if the service is bound to OData protocol version 2.
- **_O4** if the service is bound to OData protocol version 4.

For Example: /FPT/UI_SalesRep_O2, ZAPI_SalesCredit_O4

7.1.7 Behaviour Pool

Use the prefix.

- **BP_** for an ABAP class that implements the behaviour of a business object.

For Example: /FPT/BP_SalesReport, ZBP_SalesPurchaseApp

7.1.8 Handler and Saver Classes

Use the prefix.

- **LHC_** for a local handler class.
- **LSC_** for a local saver class.

Copyright © 2023 FPT All Rights Reserved.

Depending on the modularization of your behaviour implementation, you can provide the semantics of the coding in the name of the classes.

Example: LHC_SALEPURCHASE _CREATE

Example: LHC_CUSTOMER_CUD

7.2 RAP General Guideline

7.2.1 Required CDS Annotations

All view must enable metadata extension by adding below annotation (for future extension):

```
@Metadata.allowExtensions: true
```

7.2.2 Use STRICT.

Use “strict;” in program that will be delivered to customer.

(From ABAP 7.57 use can use “strict(2);”, but you need to discuss with customer or technical leader before using it)

You can read more details on strict mode here: https://help.sap.com/doc/abapdocu_latest_index_htm/latest/en-US/index.htm?file=abenc0_provider_rules_bdef.htm

7.2.3 Handling of Messages.

7.2.3.1 Message Wrapper-Class

- Message Class in RAP must implement the specialized interface IF_ABAP_BEHV_MESSAGE.
- To distinguish the classes for messages from exception classes and usual ABAP classes, their name should start with **ZCM**_ instead of ZCX_ or ZCL_.
- Sample of a Message Wrapper-class

```
CLASS ZCM_FSO_INVMSG DEFINITION
  PUBLIC
  INHERITING FROM cx_static_check
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_abap_behv_message.
    INTERFACES if_t100_message.
    INTERFACES if_t100_dyn_msg.
```


Copyright © 2023 FPT All Rights Reserved.

```
    METHODS constructor
      IMPORTING
        !im_textid    LIKE if_t100_message=>t100key OPTIONAL
        !im_serverity TYPE if_abap_behv_message=>t_severity OPTIONAL
        !im_productid TYPE zi_fso_inventory-ProductID OPTIONAL
        !im_createby  TYPE zi_fso_inventory-CreatedBy OPTIONAL.

    CONSTANTS:
      BEGIN OF c_product_not_exist,
        msgid TYPE symsgid VALUE 'ZFSO_INV',       " This is message id
        msgno TYPE symsgno VALUE '001',            " This is message number
        attr1 TYPE scx_attrname VALUE 'PRODUCTID', " Attr name of msg param
        attr2 TYPE scx_attrname VALUE '',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF   c_product_not_exist.

    CONSTANTS:
      BEGIN OF c_only_edit_by_owner,
        msgid TYPE symsgid VALUE 'ZFSO_INV',
        msgno TYPE symsgno VALUE '008',
        attr1 TYPE scx_attrname VALUE 'CREATEBY',
        attr2 TYPE scx_attrname VALUE '',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF   c_only_edit_by_owner.

    DATA:
      !productid TYPE ZI_Fso_Inventory-ProductID,
      !createby  TYPE ZI_Fso_Inventory-CreatedBy.

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.


CLASS ZCM_FSO_INVMSG IMPLEMENTATION.
  METHOD constructor ##ADT_SUPPRESS_GENERATION.
    CALL METHOD super->constructor( ).

    me->if_t100_message~t100key         = im_textid.
    me->if_abap_behv_message~m_severity = im_serverity.

    IF im_createby IS NOT INITIAL.
      me->createby = im_createby.
    ENDIF.
    IF im_productid IS NOT INITIAL.
      me->productid = im_productid.
    ENDIF.

  ENDMETHOD.

ENDCLASS.
```


Copyright © 2023 FPT All Rights Reserved.

- Sample of message use.

```
" Reject input data
APPEND VALUE #( %tky = ls_inventory-%tky ) TO failed-inventory.
" Raise message
APPEND VALUE #(
  %tky = ls_inventory-%tky
  %msg = NEW ZCM_FSO_INVMSG(
           im_textid = ZCM_FSO_INVMSG=>C_ONLY_EDIT_BY_OWNER
           im_serverity = if_abap_behv_message=>severity-error
           im_createby  = ls_inventory-CreatedBy ) ) TO reported-inventory.
```

7.2.3.2 Show messages in Validations with Draft and without Draft

- %state_area must be clear before displaying messages
- %state_area must be set if you are showing validation messages in Draft-scenarios.
- %element-<field_name> should be set with if_abap_behv=>mk-on, so that user can just click on the message to go to the incorrect field

For example:

```
" Clear state message
APPEND VALUE #( %tky = ls_inventory-%tky
                %state_area = 'PRODUCTID' ) TO reported-inventory.

" Reject input data
APPEND VALUE #( %tky = ls_inventory-%tky ) TO failed-inventory.

" Raise messages
APPEND VALUE #(
  %tky = ls_inventory-%tky
  %msg = NEW zcm_fso_invmsg (
         im_textid = zcm_fso_invmsg=>c_product_not_exist
         im_serverity  = if_abap_behv_message=>severity-error
         im_productid  = ls_inventory-ProductID )
  %element-ProductID = if_abap_behv=>mk-on
  %state_area = 'PRODUCTID' ) TO reported-inventory.
```

7.2.4 Passing Parameter to RAP Actions.

Following steps below to pass data to parameter in RAP Actions.

You can also use this to create Popup for RAP Action in Fiori Element.

- STEP1: Create an ABSTRACT ENTITY.

```
@EndUserText.label: 'Param change data SO'
```


Copyright © 2023 FPT All Rights Reserved.

```
define abstract entity ZD_PARAM_CHANGE_DATA_SO {
  pricing_data  : abap.char( 8 );
}
```

- STEP2: Create ACTION that use ABSTRACT ENTITY as parameters.

```
action UpdateData parameter ZD_PARAM_CHANGE_DATA_SO result [1] $self;
```

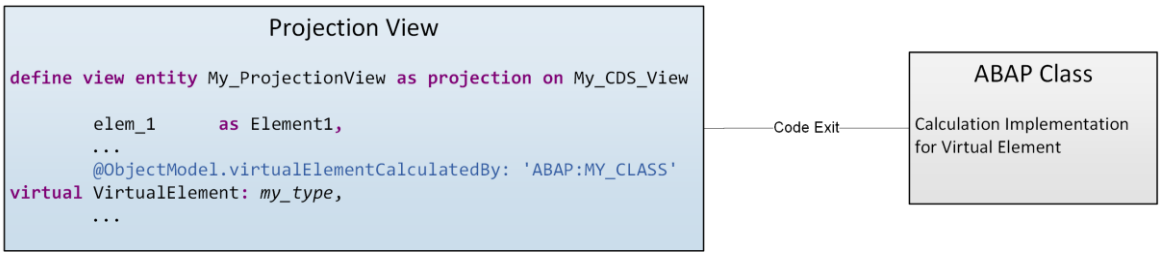
- STEP3: Implement that ACTION in Behavior Pool.

```
METHODS UpdateData FOR MODIFY
  IMPORTING it_keys FOR ACTION ZCR_SalesOrder~UpdateDate RESULT result.
……
METHOD UpdateData.
*  Process data
   LOOP AT it_keys ASSIGNING FIELD-SYMBOL(<lfs_keys>).
      (update data using values from <lfs_keys>-%param)
   ENDLOOP.
ENDMETHOD.
```

7.2.5 Using Virtual Elements in CDS Projection Views

Virtual elements are used if field elements are not provided as part of the original persistence data model but can be calculated using ABAP.

They are defined at the level of CDS projection views as additional elements within the SELECT list.



Add the interface IF_SADL_EXIT_CALC_ELEMENT_READ to the public section of your calculation class and add the two method implementations

```
CLASS ZCL_DAYS_TO_FLIGHT DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_sadl_exit_calc_element_read.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS ZCL_DAYS_TO_FLIGHT IMPLEMENTATION.
```


Copyright © 2023 FPT All Rights Reserved.

```
  METHOD if_sadl_exit_calc_element_read~get_calculation_info.
      IF iv_entity <> 'ZC_BOOKING'.
            RAISE EXCEPTION TYPE /dmo/cx_virtual_elements
                        EXPORTING
                  textid = zcx_virtual_elements=>entity_not_known
                  entity = iv_entity.
                  ENDIF.
      LOOP AT it_requested_calc_elements ASSIGNING field-
      symbol(<fs_calc_element>).
        CASE <fs_calc_element>.
          WHEN 'DAYSTOFLIGHT'.
            APPEND 'FLIGHTDATE' TO et_requested_orig_elements.
            * WHEN 'ANOTHERELEMENT'.
            * APPEND '' ...
          WHEN OTHERS.
            RAISE EXCEPTION TYPE zcx_virtual_elements
                  EXPORTING
                    textid = zcx_virtual_elements=>ve_not_known
                    element = <fs_calc_element>
                    entity = iv_entity.
        ENDCASE.
      ENDLOOP.
  ENDMETHOD.

  METHOD if_sadl_exit_calc_element_read~calculate.
    DATA(lv_today) = cl_abap_context_info=>get_system_date( ).
    DATA lt_original_data TYPE STANDARD TABLE OF zc_booking_proc
                                                WITH DEFAULT KEY.
    lt_original_data = CORRESPONDING #( it_original_data ).
    LOOP AT lt_original_data ASSIGNING FIELD-SYMBOL(<fs_original_data>).
      <fs_original_data>-DaysToFlight = <fs_original_data>-FlightDate -
                                                                lv_today.
    ENDLOOP.
    ct_calculated_data = CORRESPONDING #( lt_original_data ).
  ENDMETHOD.
ENDCLASS.
```

7.2.6 RAP Custom Entity and Queries

Use cases for unmanaged queries are

- The data source for an OData request is not a database table, but, for example another OData service, which is reached by an OData client proxy,
- Performance optimization with application specific handling,
- Using AMDPs with some query push-down parameters in the SQL script implementation,
- Forwarding the call to the analytical engines, or
- Enrichment of query result data on property or row level, for example when splitting rows for intermediate sums or condensing the filter result.

Copyright © 2023 FPT All Rights Reserved.

For example:

Custom Entity ZI_TRAVEL_UQ

```
@EndUserText.label: 'Custom entity for unmanaged travel query'
@ObjectModel.query.implementedBy:'ABAP: ZCL_TRAVEL_UQ'
define custom entity ZI_TRAVEL_UQ
{
  key Travel_ID   : abap.numc( 8 );
      Agency_ID   : abap.numc( 6 );
      Customer_ID : abap.numc( 6 );
      Begin_Date  : abap.dats;
      End_Date    : abap.dats;
      Booking_Fee : abap.dec( 17, 3 );
      Total_Price : abap.dec( 17, 3 );
      Currency_Code : abap.cuky;
      Status        : abap.char( 1 );
      LastChangedAt : timestampl;
}
```

Sample Implementation ZCL_TRAVEL_UQ

```
CLASS ZCL_TRAVEL_UQ DEFINITION
PUBLIC
FINAL
CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_rap_query_provider.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_travel_uq IMPLEMENTATION.
  METHOD if_rap_query_provider~select.
    TRY.
        CASE io_request->get_entity_id( ).
          WHEN 'ZI_TRAVEL_UQ'.
**query implementation for travel entity filter
            DATA(lv_sql_filter) = io_request->get_filter( )->get_as_sql_string( ).
            TRY.
                DATA(lt_filter) = io_request->get_filter( )->get_as_ranges( ).
              CATCH cx_rap_query_filter_no_range.
                "handle exception
            ENDTRY.
**CDS parameters
            DATA(lt_parameters) = io_request->get_parameters( ).
            DATA(lv_next_year) = CONV syst_datum( cl_abap_context_info=>get_system_date( ) + 365 ).
            DATA(lv_par_filter) = | BEGIN_DATE >= '{ cl_abap_dyn_prg=>escape_quotes(
                VALUE #( lt_parameters[ parameter_name = 'P_START_DATE' ]-value
                         DEFAULT cl_abap_context_info=>get_system_date( ) ) ) }'| && | AND | &&
            | END_DATE <= '{ cl_abap_dyn_prg=>escape_quotes(
                VALUE #( lt_parameters[ parameter_name = 'P_END_DATE' ]-value
                         DEFAULT lv_next_year ) ) }'| .
            IF lv_sql_filter IS INITIAL.
              lv_sql_filter = lv_par_filter.
            ELSE.
              lv_sql_filter = |({ lv_sql_filter } AND { lv_par_filter } )| .
            ENDIF.
**Build SQL search
            DATA(lv_search_string) = io_request->get_search_expression( ).
            DATA(lv_search_sql) = |DESCRIPTION LIKE
                                   '%{ cl_abap_dyn_prg=>escape_quotes( lv_search_string ) }%'|.
            IF lv_sql_filter IS INITIAL.
              lv_sql_filter = lv_search_sql.
            ELSE.
```


Copyright © 2023 FPT All Rights Reserved.

```
              lv_sql_filter = |( { lv_sql_filter } AND { lv_search_sql } )|.
            ENDIF.
**request data
            IF io_request->is_data_requested( ).
**Sample paging
              DATA(lv_offset) = io_request->get_paging( )->get_offset( ).
              DATA(lv_page_size) = io_request->get_paging( )->get_page_size( ).
              DATA(lv_max_rows) =
                      COND #( WHEN lv_page_size = if_rap_query_paging=>page_size_unlimited
                              THEN 0 ELSE lv_page_size ).
**Sample sorting
              DATA(sort_elements) = io_request->get_sort_elements( ).
              DATA(lt_sort_criteria) = VALUE string_table(
                          FOR sort_element IN sort_elements
                            ( sort_element-element_name &&
                                COND #( WHEN sort_element-descending = abap_true THEN `descending`
                                        ELSE ` ascending` ) ) ).
              DATA(lv_sort_string) =
                      COND #( WHEN lt_sort_criteria IS INITIAL THEN `primary key`
                              ELSE concat_lines_of( table = lt_sort_criteria sep = `, ` ) ).
**requested elements
              DATA(lt_req_elements) = io_request->get_requested_elements( ).
**Sample aggregate
              DATA(lt_aggr_element) = io_request->get_aggregation( )->get_aggregated_elements( ).
              IF lt_aggr_element IS NOT INITIAL.
                LOOP AT lt_aggr_element ASSIGNING FIELD-SYMBOL(<fs_aggr_element>).
                  DELETE lt_req_elements WHERE table_line = <fs_aggr_element>-result_element.
                  DATA(lv_aggregation) = |{ <fs_aggr_element>-
aggregation_method }( { <fs_aggr_element>-input_element } ) as { <fs_aggr_element>-
result_element }|.
                  APPEND lv_aggregation TO lt_req_elements.
                ENDLOOP.
              ENDIF.
              DATA(lv_req_elements) = concat_lines_of( table = lt_req_elements
                                                       sep   = `, ` ).
**** Sample grouping
              DATA(lt_grouped_element) = io_request->get_aggregation( )->get_grouped_elements( ).
              DATA(lv_grouping) = concat_lines_of( table = lt_grouped_element
                                                   sep   = `, ` ).
**select data
              DATA lt_travel_response TYPE STANDARD TABLE OF zi_travel_uq.

              SELECT (lv_req_elements) FROM ztravel
                WHERE (lv_sql_filter)
                GROUP BY (lv_grouping)
                ORDER BY (lv_sort_string)
                INTO CORRESPONDING FIELDS OF TABLE @lt_travel_response
                OFFSET @lv_offset UP TO @lv_max_rows ROWS.
**fill response
              io_response->set_data( lt_travel_response ).
            ENDIF.
**request count
            IF io_request->is_total_numb_of_rec_requested( ).
**select count
              SELECT COUNT( * ) FROM ztravel
                WHERE (lv_sql_filter)
                INTO @DATA(lv_travel_count).
**fill response
              io_response->set_total_number_of_records( lv_travel_count ).
            ENDIF.
          WHEN `ZI_BOOKING_UQ`.
**query implementation for booking entity
        ENDCASE.
      CATCH cx_rap_query_provider.
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```


Copyright © 2023 FPT All Rights Reserved.

7.2.7 Other RAP Guidelines form Standard Document of SAP

Reference: https://help.sap.com/doc/3750bcdf7b8045e18f1b759e6d2b000b/Cloud/en-US/ABAP_RESTful_Programming_Model_EN.pdf

Copyright © 2023 FPT All Rights Reserved.